

IAP - Introduction à l'Algorithmique et à la Programmation

Equipe pédagogique

*Marie-José Caraty, Julien Rossit, Camille Kurtz,
Jacques Alès-Bianchetti, Eloi Keita*

Cours n°6

Les fondamentaux de la programmation impérative

Analyse du projet (2/2)

http://www.tutorialspoint.com/c_standard_library/

5. CYCLE DE DEVELOPPEMENT LOGICIEL

Quelques dates qui vous intéresseront...

Date de remise des dossiers de développement

Le **lundi 6 novembre** au secrétariat (retard pénalisé)

Date des recettes

La **semaine du 13 novembre**

1^{ère} semaine du module SDA-Structures de Données et Algorithmes
en séance de SDA3 (2^{ème} TP de SDA)

Cette semaine (du 16 octobre)

Les deux séances de TP (IAP2 et IAP3) sont dédiées au projet

5. CYCLE DE DEVELOPPEMENT LOGICIEL

La recette...

Date de remise des dossiers de développement

Le **lundi 6 novembre** au secrétariat (retard pénalisé)

Date des recettes

La **semaine du 13 novembre**

1^{ère} semaine du module SDA-Structures de Données et Algorithmes
en séance de SDA3 (2^{ème} TP de SDA)

Cette semaine (du 16 octobre), les deux séances de TP (IAP2 et IAP3)
sont dédiées au projet

2. RECETTE DE L'APPLICATION

Le passage de la recette

Recette de l'application

Validation par un observateur (client/votre enseignant de TP)
du Sprint de plus haut niveau atteint en phase
de développement

Recette de l'application

Préparation à la recette

Préparez un projet Windows de votre application sur une machine de votre choix (machine de l'IUT ou portable personnel).

Recette

En présence de votre enseignant de TP, vous testerez votre programme avec un JDT de recette (`inSPn.txt`) correspondant au niveau n ($1 \leq n \leq 5$) le plus haut que vous avez validé en phase de développement.

Ce JDT vous sera donné par votre enseignant

Vous nommerez `run.txt` le fichier de sortie.

Si votre fichier `run.txt` est identique au fichier résultat de référence `ref.txt` (donné par votre enseignant), votre logiciel est accepté : le `Sprint#n` est validé.

Sinon votre logiciel est refusé, vous avez 5 minutes pour corriger vos erreurs.

A la suite de ce délai, si vous ne passez toujours pas la recette, vous aurez à valider le `Sprint#n-1`.

Rem : utilisation d'un comparateur de fichier (`diff`) pour comparer les fichiers

Le problème posé



Programmer un interpréteur de 9 commandes comprenant 8 commandes de manipulation d'une table de données générique

- 1 commande de sortie de l'interpréteur
- 8 commandes de manipulation de table

Les commandes sont codées sous formes de chaînes de caractères et entrées par l'utilisateur/par redirection du fichier d'entrées

Généricité de la table : 4 types de données sont considérés (exclusivement) TEXT (chaîne de caractères), INT (entier), FLOAT (réel) et DATE (format "jj/mm/aaaa")

Toutes les données des JDT (jeux de test) de recette **sont nominales** c'est-à-dire sans erreur (e.g., de type)

Coder un interpréteur de commandes

Interpréteur des 9 commandes

codées sous forme de chaînes de caractères

- Exit (1. Sprint#1)
- Create_table (2. Sprint#1)
- Afficher_schema (3. Sprint#1)
- Insert_enregistrement (4. Sprint#2)
- Afficher_enregistrements (5. Sprint#2)
- Delete_enregistrement (6. Sprint#3)
- Afficher_enregistrement (7. Sprint#3)
- Delete_table (8. Sprint#4)
- Select_enregistrement (9. Sprint#5)

Spécifications

(1/2)

Toute spécification données doit être suivie sous peine de pénalisation (cf. 1^{er} cours Projet - semaine 4)

Spécifications de sortie données

Cf. Transparents 3 à 7

Objectif : *Unification des sorties attendues*

Spécification des constantes utilisées

cf. Transparents 8 à 9

Objectif : *Eviter les nombres magiques par nommage et dimensionnement des constantes utilisées*

Nombre magique : sont considérés comme nombre magique toute constante numérique littérale différente de 0 et 1

L'interpréteur des commandes

Les commandes sont codées sous forme de chaînes de caractères et entrées – en utilisant l'entrée standard
– ou par redirection d'un fichier texte sur l'entrée standard

Neuf commandes devront être reconnues par l'interpréteur :

1. Create_table

Commande la création d'une table de données et la définition de son schéma de données :

une ligne composée de la chaîne de caractères "Create_table" suivie du nom de la table, du nombre de champs, et pour chaque champ : du nom du champ et de son type.

Ex : *Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT*

Spécification : Si la table est déjà créée, la commande affichera "table existante" sinon "table inconnue".

Les constantes du problème – Leur déclaration (1/2)

La déclaration des tableaux statiques nécessite la connaissance de leur taille

On est amené à dimensionner le problème et souvent à sur-dimensionner la taille des tableaux

Quelques constantes du problème

- le nombre maximum de champs d'une table (**maxChamps**) peut varier, il est borné à **25**
- Le nombre maximum d'enregistrement d'une table est borné à **100**
- la taille maximale d'une chaîne de caractères considérée est **30**
- la taille maximale d'une ligne de commande (comprenant la commande et ses paramètres) considérée est **80**

Spécifications

(2/2)

Autre spécification
cf. Transp. 23

« Les champs d'information d'une commande seront lus dans la fonction correspondante »

Objectif : *Structuration du code par unification du traitement des commandes*

Sprint#1 – Prototypage des fonctions Transp. 23

Spécification : les champs d'information d'une commande seront lus dans la fonction correspondante

Sprint#1

Prototype des fonctions

```
void create_table(Table* t);
```

```
void afficher_schema(const Table* t);
```

```
// void create_table(Table t);
// Prototype non optimisé en raison de la copie
// du paramètre effectif dans le paramètre formel
```

La structuration des données est obligatoire

Pénalisation en cas de non structuration : **0/20**

Rappel

Une structure de données permet de regrouper les données liées à un type d'information

Raison majeure de la nécessité de structuration des données

Les fonctions, leur prototypage et leur appel

Passer en paramètre une variable de type structuré permet un accès (en lecture ou écriture suivant le type de passage [in], [out] ou [in out]) à chacun de ses champs

Exemple : `afficher (Table* t);`
`afficher (const Table* t); //mieux (cf. cours 5)`

Champ

nom chaîne
 type chaîne

Table

nom chaîne
 schema Champ
 nbChamps unsigned char

```
// Affichage des caractéristiques de la table de donnée
// i.e. nom de la table et chacun de ses champs (nom et type)
void afficher (const Table* t);
```

Les structures de données peuvent évoluer en fonction des sprints

Data

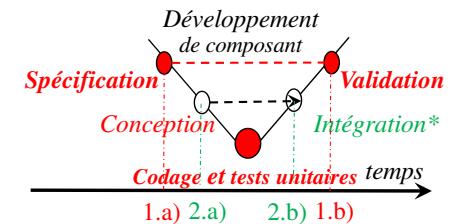
data Chaîne (lmot)
 data

Enregistrement

colonne Data (maxChamps)
 colonne

Table

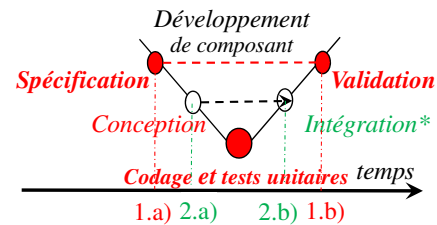
nom Chaîne (lmot)
 schema Champ (maxchamps)
 enrg Enregistrement (maxEnr)
 nbChamps int
 nbEnrg int



Le développement « agile » est l'un des grands principes du Génie logiciel

Dépasse le cadre de la programmation/codage
 Méthodologie de développement « sûr »

Développement agile



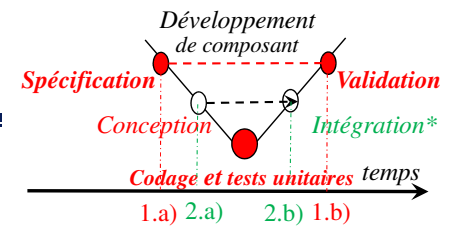
Choix d'un développement agile

Développement par Sprints,
chacun représentant un incrément de fonctionnalité
de l'application

Cinq sprints sont définis et pour chacun des Sprints :

- 1) une analyse fonctionnelle
- 2) une spécification
- 3) un codage correspondant
- 4) un test de validation

Validation d'un Sprint



Principe de validation d'un **Sprint#n** à partir :

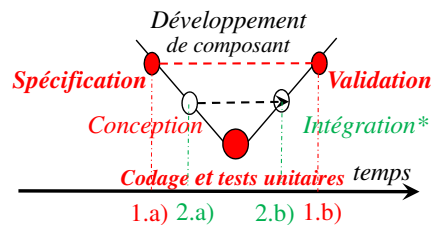
Jeu de Données de Test (JDT) : `inSpn.txt`
et des sorties attendues : `outSPn.txt`

L'exécution de l'exécutable du Sprint#n par redirection

- des entrées à partir de `inSpn.txt`
- des sorties vers `run.txt`

Si `outSPn.txt` coïncide avec `run.txt`
le Sprint#n est 0-défaut, il est validé

Validation d'un Sprint



A maîtriser

La redirection des entrées et sorties (cf. Aide à projet sur COMMUN).

PS : Vous pouvez glisser le nom de l'exécutable
(nommé "Application" dans le dossier <debug> sous Visual Studio)
dans la fenêtre de commande DOS (cmd)

De même, vous pourrez glisser le fichier "inSp#.txt"
(où # est le n° de Sprint) dans cette fenêtre

Sprint#5 – Le type Date

(7/8)

Les dates sont introduites au Sprint#5

Elles constituent le dernier type (Date) qui sera manipulé
dans les JDT de recette.

Dans les JDT

Les dates seront normalisées sous la forme d'une chaîne
de caractères "jj/ mm/aa" (e.g., "16/10/2017")

Vous pourrez définir dans votre code le type structuré (Date)
correspondant à une date (jour, mois et année)

(7/8)

1	6	/	1	0	/	2	0	1	7
0	1	2	3	4	5	6	7	8	9

char *strncpy(char *dest, const char *src, size_t n)
Copie *n* caractères de la chaîne pointée par *src* dans la chaîne *dest*.

```
char aux[lMot];
// Copie de la chaîne repérée par le pointeur mot et de longueur 2
// dans la chaîne aux, puis conversion de aux en entier
char jour= atoi(strncpy(aux, mot, 2));
// Utilisation de l'arithmétique des pointeurs
char mois= atoi(strncpy(aux, mot+3, 2)); //mot+3 pointe l'index 3
//même principe pour l'année
```

Eléments de qualité d'un projet

- **Architecture logicielle**
- **Structuration des données du programme**
- **Analyse des fonctionnalités/fonctions**
- **Code**
- **Documentation**
- **Livrable (dossier de programmation)**

- **Notion différée : Architecture logicielle**

IOCCC-International Obfuscated C Code Contest
https://fr.wikipedia.org/wiki/International_Obfuscated_C_Code_Contest

IOCCC' 1987 – The winner was...

IOCCC-International Obfuscated C Code Contest
[https://fr.wikipedia.org/wiki/International Obfuscated C Code Contest](https://fr.wikipedia.org/wiki/International_Obfuscated_C_Code_Contest)

IOCCC' 1998 – The winner was...

[illegible]

Dossier de développement logiciel

☐ Respect des spécifications

Respect des spécifications données pour le dossier de développement logiciel (livrable)

Respect du cahier des charges de présentation

Présence dans le dossier a) une page de garde indiquant le nom et le groupe des membres du binôme, l'objet du dossier, illustration, b) une table des matières de l'ensemble du dossier (incluant les annexes) avec la pagination de toutes les rubriques, la pagination est continue du début à la fin du dossier c) présentation de l'application, e) organisation des tests, f) bilan de validation des tests de développement, g) bilan de projet. Brochez vos dossiers.

☐ Présentation de l'application

L'art de synthétiser le projet dans un format donné (1 page). On devra trouver dans cette présentation son rôle fonctionnel (ce que fait l'application), ses entrées et ses sorties (au moins pour le Sprint validé).

Qualité de code

(1/2)

☐ Lisibilité du code

Code lisible

Code parfaitement indenté (tabulation)

Ligne de code limitée à 80 colonnes

☐ Documentation

Du fichier source (en début de fichier : nom de fichier, nom et groupe(s) des auteurs, date de création)

Des types structurés et de leurs champs

Des variables importantes

Des fonctions :

rôle de la fonction, des paramètres formels, leur mode ([in], [out] et [in-out])

rôle du paramètre de retour éventuel

```
/*
 * Calcul du maximum de deux valeurs
 * [in] x, 1ère valeur
 * [in] y, 2ème valeur
 * [out] mx, le maximum
 */
void max(int x, int y, int* mx);
```

Qualité de code

(2/2)

☐ Commentaires dans le code

Savoir commenter les parties « complexes » du code

☐ Absence de nombres magiques

Absence de littéraux (autre que 0 et 1), utilisation des constantes ou de #define motivée par des changements possibles des constantes considérées

☐ Longueur des fonctions

Le main doit être court (à très court, quelques lignes).

Un main est à un niveau macroscopique (constitué d'appels de fonction).

Un main trop long traduit un manque au niveau de l'analyse (des fonctions auraient dû être conçues et être appelées par le main).

Par exemple, on n'y développe pas une interface (e.g., un menu).

Une fonction ne doit pas dépasser une vingtaine de lignes. Une fonction trop longue indique souvent un manque d'analyse fonctionnelle et s'accompagne de redondance : dans ce cas, on relit et on analyse le code en visant un niveau macroscopique (de l'algorithme) pour introduire les fonctions (à coder) et qui seront à appeler (la longueur du code diminuera).

Les tests

☐ Code redondant

Introduire la/les fonction(s) qui généralisent les traitements.

☐ Tests

D'une manière générale, vous devez préciser votre stratégie de test : comment sont organisés vos tests.

Un test comprend : un objectif (ce que l'on veut tester), un JDT (jeu de test), un résultat attendu (résultat de référence), un résultat (**trace d'exécution** de votre programme) et un **bilan de validation** (ce que valide le test). Il faut rédiger cette partie.

Dans le rapport, on doit trouver une partie de « **Bilan de validation des tests** » où on résume tous les tests et ce qu'ils valident.

But : vérifier votre compréhension des tests.

Des JDT « personnels » servent à améliorer la couverture des tests (non demandés)

☐ Bilan de projet

Retour d'expérience. Les difficultés rencontrées, ce qui est réussi, ce qui peut être amélioré

Evaluation des projets – Barème indicatif

[15 pts] NS	Note de Sprint (du plus haut niveau #n) validé lors de la recette
[1 pt] MOD	Modulation d'un Sprint de niveau n validé relativement au source du sprint#n+1 (si présent dans le dossier)
[1 pt] PA	Présentation de l'application
[1 pt] BV	Bilan de validation
[1 pt] BP	Bilan de projet
[2 pts] QC	Qualité du code
[-1 pt] PN	Pénalités de non respect des spécifications (cf. texte du projet) Ex. N° de groupe absent sur la page de garde, pagination absente..

Barème indicatif

NS [/15 points] Barème établi sur le Sprint de plus haut niveau validé à la recette
 Sprint 5 : 15 points
 Sprint 4 : 12 points
 Sprint 3 : 09 points
 Sprint 2 : 06 points
 Sprint 1 : 03 points
 Sprint 1 non atteint : 0 point

ATTENTION 0/20 si le dossier de développement n'est pas rendu
 0/20 si le code n'est pas présent dans le dossier